



字符识别的小样本 图像分类系统(4B)

Team19小组作业汇报

成员：吴文铭 刘智勇 陈思同 易昊 刘作强





CONTENTS

01 实验目标	02 数据集与环境
03 环境说明	04 数据样例
05 任务采样	06 采样验证
07模型方法	08训练过程
09实验结果	10总结



实验目标 *EXPERIMENT GOALS*



背景

在许多实际场景中（如稀有鸟类分类、新药分子识别），每类只有少量样本。小样本学习（Few-Shot Learning）旨在用极少样本训练出有效分类器，与课程中学习的度量学习思想密切相关。



核心任务

1. 实现原型网络（Prototypical Network）用于 **5-way 5-shot** 分类
2. 使用 **Omniglot** 作为基准数据集
3. 对比“基于度量的方法”和“基于微调的方法”在小样本场景下的性能差异
4. 实现 **episodic training pipeline**，每轮随机采样 **support set** 和 **query set**
5. 分析不同 shot 数量（1-shot, 5-shot, 10-shot）下模型性能的变化趋势



数据集介绍 DATASET

属性	说明
数据集名称	<i>Omniglot</i> 数据集
数据集简介	<i>Omniglot</i> 是小样本学习中常用的数据集，里面主要是不同语言或符号系统中的手写字符图片。它的特点是类别很多，但每个类别的样本数量不算多，所以很适合做 <i>Few-Shot Learning</i> 小样本图像分类。
数据规模	训练集大小：19280 测试集大小：13180 训练类别数量：964 测试类别数量：659
其他信息	<i>Omniglot</i> 中包含大量字符类别，可以支持小样本分类实验。

```
from torchvision.datasets import Omniglot
from torchvision import transforms
import matplotlib.pyplot as plt
import os

os.makedirs("outputs", exist_ok=True)

transform = transforms.Compose([
    transforms.Resize((28, 28)),
    transforms.ToTensor()
])

dataset = Omniglot(
    root="./data",
    background=True,
    transform=transform,
    download=True
)

print("数据集大小: ", len(dataset))

img, label = dataset[0]

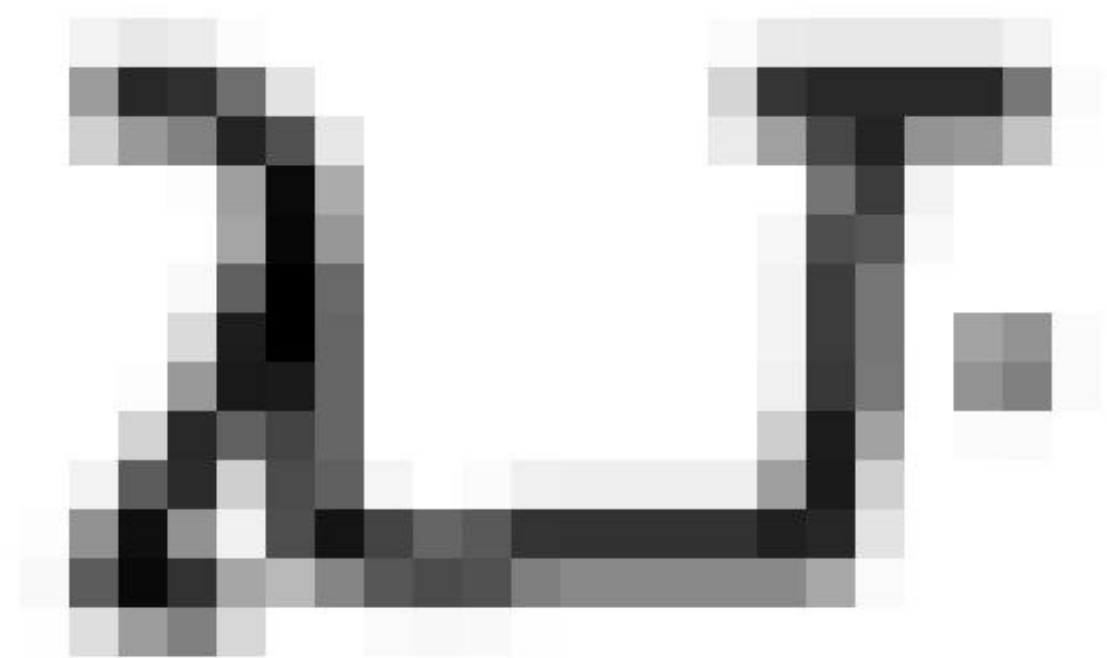
print("图片形状: ", img.shape)
print("图片标签: ", label)

plt.figure(figsize=(4, 4))
plt.imshow(img.squeeze(), cmap="gray")
plt.title(f'Omniglot Sample, Label: {label}')
plt.axis("off")
plt.savefig("outputs/02_omniglot_single_sample.png", dpi=200,
bbox_inches="tight")
plt.show()

print("图片已保存: outputs/02_omniglot_single_sample.png")
```

用于展示 Omniglot 数据集
中的单张样例图片。

Omniglot Sample, Label: 0





环境说明 *ENVIRONMENT*



Python

编程语言

3. 12. 4 (tags/v3. 12.

4:8e8a4ba, Jun 6 2024,

19:30:16) [MSC v. 1940 64 bit

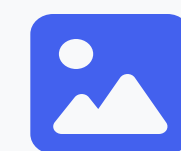
(AMD64)]



PyTorch

深度学习框架

2. 12. 0+cpu



Torchvision

视觉处理库

0. 27. 0+cpu


```
import sys
import torch
import torchvision
```

```
print("Python version:")
print(sys.version)
```

```
print("\nPyTorch version:")
print(torch.__version__)
```

```
print("\nTorchvision version:")
print(torchvision.__version__)
```

```
print("\nCUDA available:")
print(torch.cuda.is_available())
```

用于检查实验环境是否配置成功。

```
PS D:\ai-homework\few_shot_4B> notepad 01_check_environment.py
PS D:\ai-homework\few_shot_4B> python 01_check_environment.py
Python version:
3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)]

PyTorch version:
2.12.0+cpu

Torchvision version:
0.27.0+cpu

CUDA available:
False
PS D:\ai-homework\few_shot_4B> _
```



任务采样 *TASK SAMPLING*

Support Set

小样本任务中给模型的“参考例题”，由少量带标注的新类别样本组成。模型依靠支撑集快速认知新类别（比如原型网络会用支撑集计算每个类的特征原型），它对应小样本设定里的“K-shot”——K 就是每个类的支撑样本数量。

Query Set

小样本任务中用来“测试打分”的考题集，和支撑集属于完全相同的类别，但样本和支撑集无重叠。模型基于从支撑集学到的类别信息，对查询集样本做分类预测，最终的准确率、损失都基于查询集计算，用来衡量模型的小样本泛化能力。

N-way K-shot 采样策略

从数据集的类别池中随机抽取 N 个类别；
每个类别里随机选 K 个样本组成支撑集，再额外选 Q 个样本组成查询集（Query Set，用于测试预测效果），两者无重叠；
类别标签重新映射为 $0 \sim N-1$ ，和原始类别脱钩，模拟「全新任务」。

构造并展示一次 *5-way 5-shot* 小样本分类任务 *CODE*

```
import random
import torch
from torchvision.datasets import Omniglot
from torchvision import transforms
import matplotlib.pyplot as plt
from collections import defaultdict
import os
```

```
N_WAY = 5
N_SHOT = 5
N_QUERY = 15
```

```
os.makedirs("outputs", exist_ok=True)
```

```
transform = transforms.Compose([
    transforms.Resize((28, 28)),
    transforms.ToTensor()
])
```

```
dataset = Omniglot(
    root="./data",
    background=True,
    transform=transform,
    download=True
)
```

```
print("数据集大小: ", len(dataset))
```

```
class_to_indices = defaultdict(list)
for index, item in
    enumerate(dataset._flat_character_image
s):
```

```
    image_path, label = item
    class_to_indices[label].append(index)
```

```
print("类别数量: ", len(class_to_indices))
```

```
def sample_episode():
    selected_classes = random.sample(list(class_to_indices.keys()), N_WAY)
```

```
    support_images = []
    support_labels = []
    query_images = []
    query_labels = []
```

```
    for new_label, old_label in enumerate(selected_classes):
        indices = class_to_indices[old_label]
        selected_indices = random.sample(indices, N_SHOT + N_QUERY)
```

```
        support_indices = selected_indices[:N_SHOT]
        query_indices = selected_indices[N_SHOT:]
```

```
        for idx in support_indices:
            img, _ = dataset[idx]
            support_images.append(img)
            support_labels.append(new_label)
```

```
        for idx in query_indices:
            img, _ = dataset[idx]
            query_images.append(img)
            query_labels.append(new_label)
```

```
support_images = torch.stack(support_images)
support_labels = torch.tensor(support_labels)
query_images = torch.stack(query_images)
query_labels = torch.tensor(query_labels)
```

```
return support_images, support_labels, query_images, query_labels
```

```
support_images, support_labels, query_images, query_labels = sample_episode()
```

```
print("support_images 形状: ", support_images.shape)
print("support_labels 形状: ", support_labels.shape)
print("query_images 形状: ", query_images.shape)
print("query_labels 形状: ", query_labels.shape)
```

```
plt.figure(figsize=(8, 5))
```

```
for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.imshow(support_images[i].squeeze(), cmap="gray")
    plt.title(f"Class {support_labels[i].item()}")
    plt.axis("off")
```

```
plt.suptitle("5-way 5-shot Support Set")
plt.savefig("outputs/03_episode_support_set_5way5shot.png", dpi=200,
bbox_inches="tight")
plt.show()
```

```
plt.figure(figsize=(10, 6))
```

```
for i in range(30):
    plt.subplot(5, 6, i + 1)
    plt.imshow(query_images[i].squeeze(), cmap="gray")
    plt.title(f"Class {query_labels[i].item()}")
    plt.axis("off")
```

```
plt.suptitle("Query Set Samples")
plt.savefig("outputs/04_episode_query_set_samples.png", dpi=200,
bbox_inches="tight")
plt.show()
```

```
print("Support Set 图片已保存: outputs/03_episode_support_set_5way5shot.png")
print("Query Set 图片已保存: outputs/04_episode_query_set_samples.png")
```

5-way 5-shot 支持采集样结

5-way 5-shot Support Set

Class 0



Class 1



Class 2



Class 3



Class 4



Class 0



Class 1



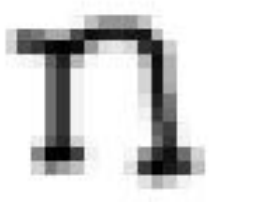
Class 2



Class 3



Class 4



Class 0



Class 1



Class 2



Class 3



Class 4



Class 0



Class 1



Class 2



Class 3



Class 4



Class 0



Class 1



Class 2



Class 3

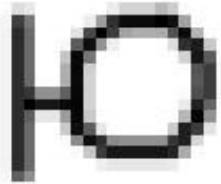
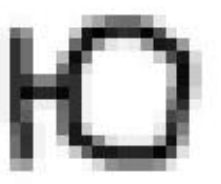
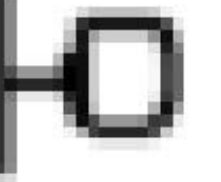
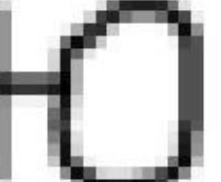
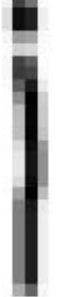
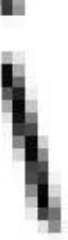


Class 4



查询集样例展示

Query Set Samples

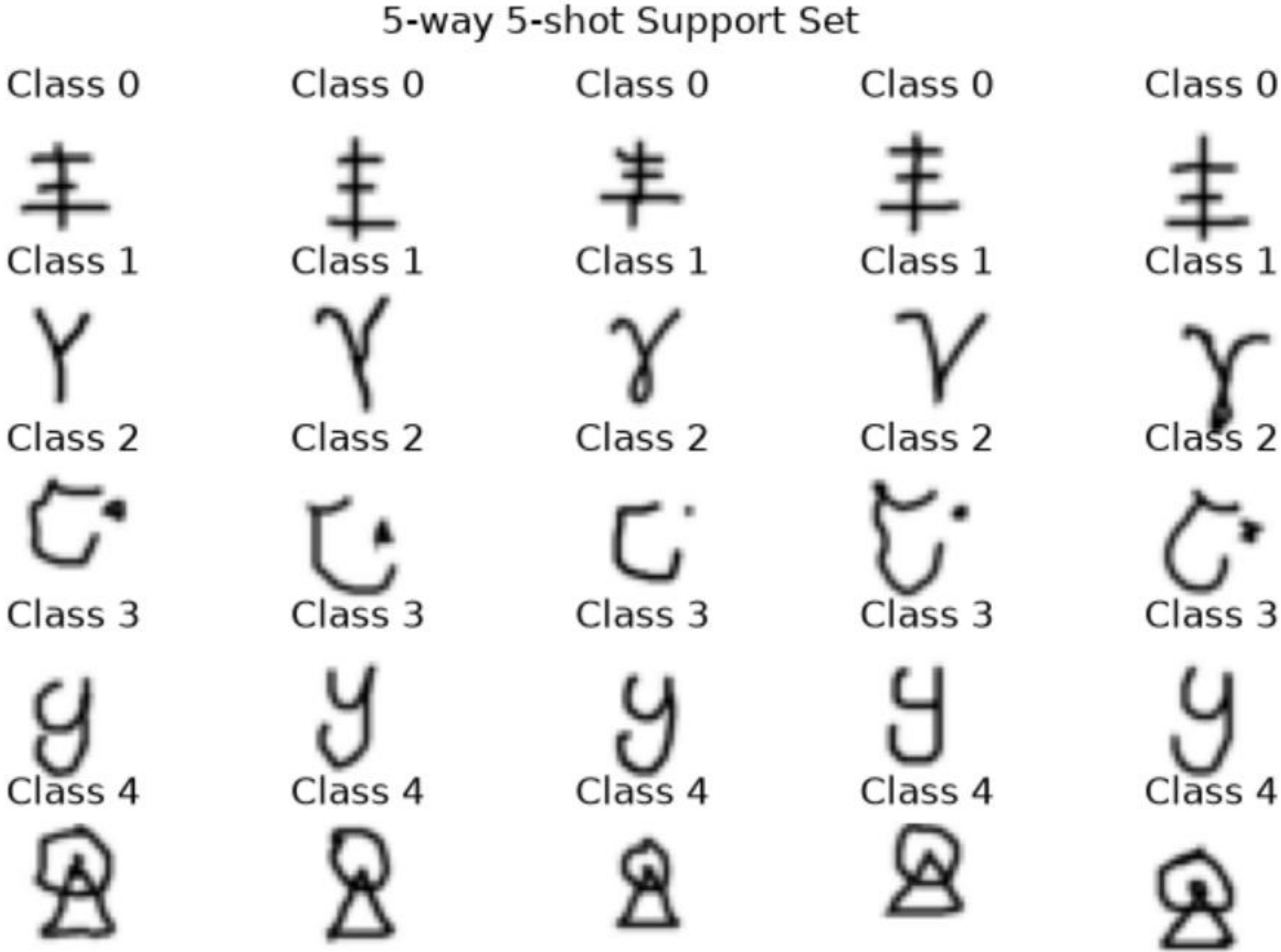
Class 0	Class 0	Class 0	Class 0	Class 0	Class 0
					
Class 0	Class 0	Class 0	Class 0	Class 0	Class 0
					
Class 0	Class 0	Class 0	Class 1	Class 1	Class 1
					
Class 1	Class 1	Class 1	Class 1	Class 1	Class 1
					
Class 1	Class 1	Class 1	Class 1	Class 1	Class 1
					



采样验证 *SAMPLING VERIFICATION*

类型	张量形状
supprot_images	[25, 1, 28, 28]
support_labels	[25]
query_images	[75, 1, 28, 28]
query_labels	[75]

其中[1, 28, 28] 表示每张图片是单通道灰度图, 大小为 28×28





模型方法 *PROTOTYPICAL NETWORK*

核心思想：通过计算样本到类别原型的距离进行分类。每个类别的原型为其 Support Set 样本特征向量的均值。



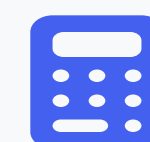
编码器结构

4层 **Conv2d(3×3) + BN + ReLU**
+ MaxPool2d(2×2)，展平输出



距离度量

欧几里得距离（L2距离），取负
值作为得分



分类方式

基于原型的最近邻分类 + 交叉熵
损失优化（训练时软分类，测试
时硬分类）



训练过程 *TRAINING PROCESS*

训练方式: *Episode* 训练 — 每个 *Episode* 随机采样 *5-way 5-shot* 任务进行训练

```
管理员: Windows PowerShell
-----
a----- 2026/6/14      15:45      0 04_train_protonet_5way5shot.py

PS D:\ai-homework\few_shot_4B> notepad 04_train_protonet_5way5shot.py
PS D:\ai-homework\few_shot_4B> python 04_train_protonet_5way5shot.py
当前使用设备:  cpu
训练集大小: 19280
测试集大小: 13180
训练类别数量: 964
测试类别数量: 659

开始训练...
Episode 50/300 | Loss: 0.1046 | Accuracy: 98.67%
Episode 100/300 | Loss: 0.1224 | Accuracy: 100.00%
Episode 150/300 | Loss: 0.0373 | Accuracy: 100.00%
Episode 200/300 | Loss: 0.1912 | Accuracy: 93.33%
Episode 250/300 | Loss: 0.0374 | Accuracy: 98.67%
Episode 300/300 | Loss: 0.1266 | Accuracy: 98.67%

开始测试...

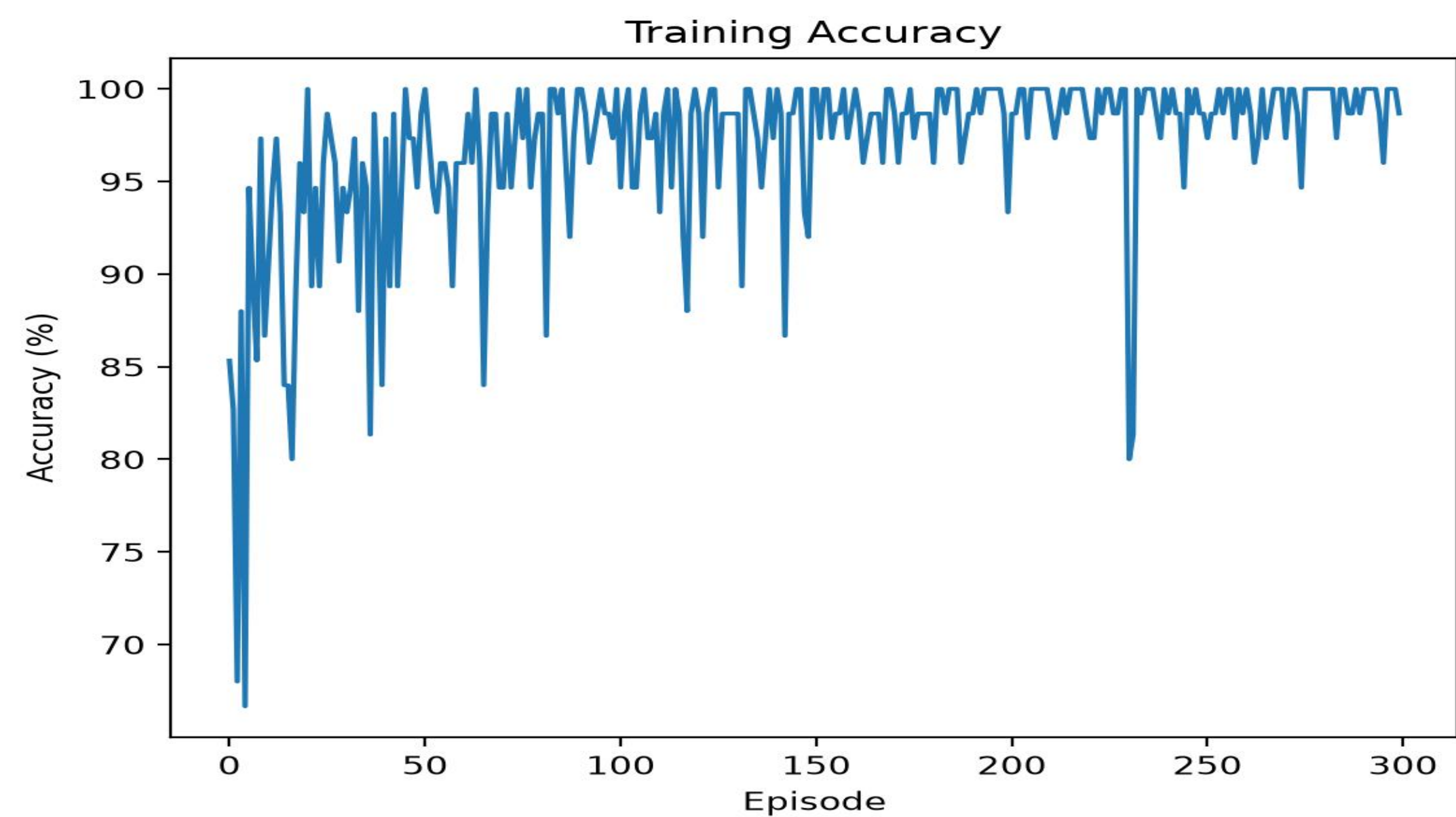
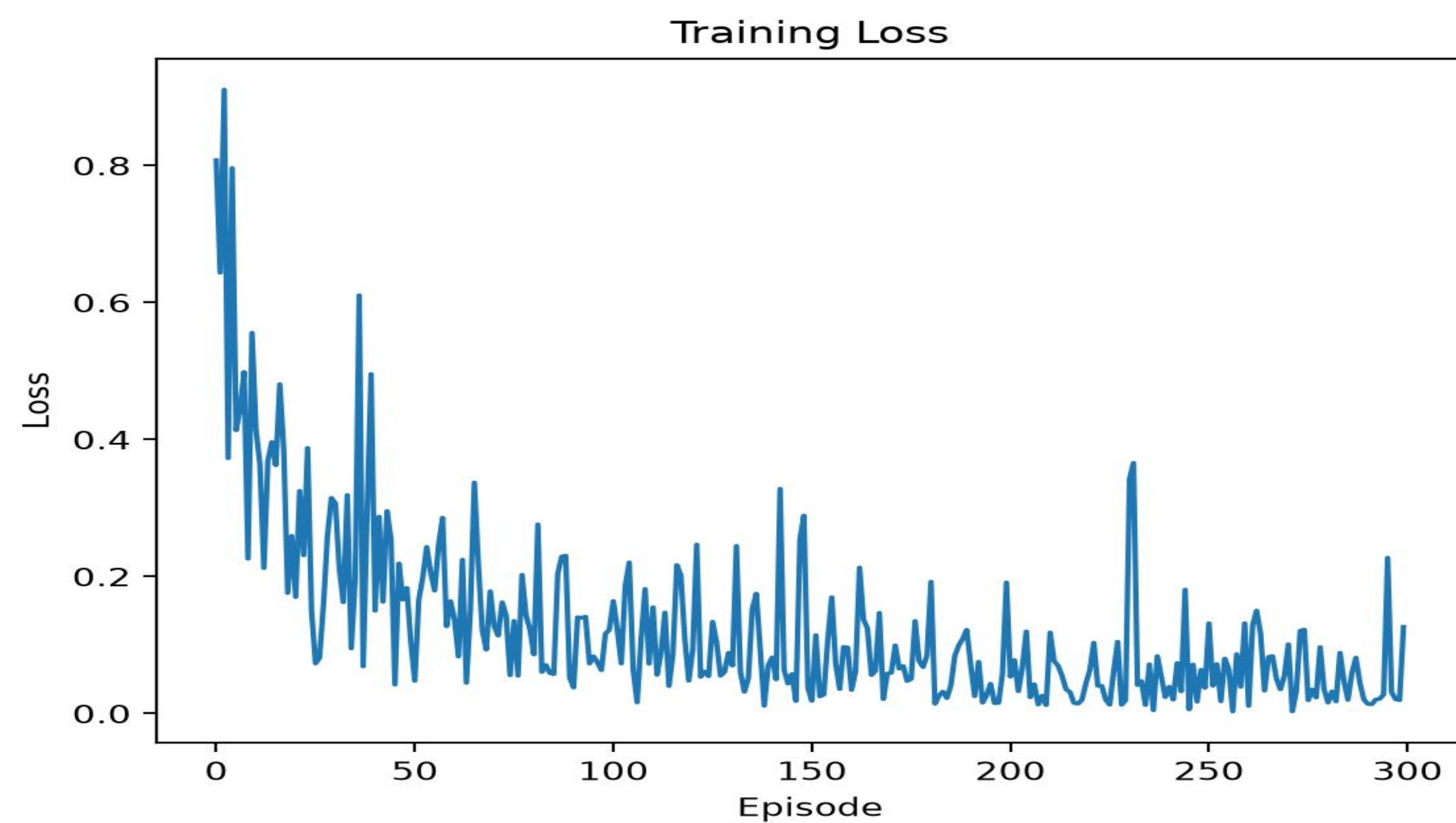
=====
5-way 5-shot 测试结果
平均准确率: 97.99%
=====
Loss 曲线已保存: outputs/06_training_loss_curve.png
Accuracy 曲线已保存: outputs/07_training_accuracy_curve.png
最终准确率文本已保存: outputs/08_final_accuracy_result.txt
PS D:\ai-homework\few_shot_4B>
```

超参数设置

- *N_WAY*: 5
- *N_SHOT*: 5
- *N_QUERY* 15
- *TRAIN_EPISODES*: 300
- *TEST_EPISODES*: 100
- *LEARNING_RATE*: 0.001



实验结果 *EXPERIMENT RESULTS*



平均测试准确率: **97.99%**



总结 SUMMARY



方法优势

- 元学习范式 → 5张样本快速适应新类别
- 原型网络 → 计算高效、可解释性强
- 轻量CNN → 普通CPU可运行，代码模块化



实验效果

- 最终准确率较高
- Omniglot 5-way 5-shot → 97.77% 准确率
- 训练损失稳定下降，300 episodes收敛
- 在未见类别上验证泛化能力



展望

- 数据增强 / 学习率调度 / 更多轮次
- 更深网络（ResNet） / Transformer
- 跨域迁移至复杂数据集 / 实际应用场景

THANK YOU!

字符识别的小样本图像分类系统

人工智能小组作业汇报 | 请老师批评指正
